



Course ID and Title: CSCI 599 Agentic AI

Units: 4

Term—Day—Time: Fall 2026 — TueTh — 7:00-8:50 pm

Location: SGM 124

Instructor: Marco Papa

Office: GCS 302E

Office Hours: 1:00-4p.m. Thursday

Contact Info: papa@usc.edu

Teaching Assistant: Faith, Baca

Contact Info: faithbac@usc.edu

Teaching Assistant: Joo, Kevin

Contact Info: khjoo@usc.edu

Catalogue Description

Design, implement, and deploy LLM-powered autonomous agents using production frameworks, interoperability protocols, and hybrid cloud-local inference, emphasizing safety evaluation and real-world deployment.

Course Description

This course examines the design, development, and deployment of agentic AI systems — autonomous agents powered by large language models (LLMs) with capabilities including planning, memory, tool use, and multi-step reasoning. Students analyze architectural patterns such as ReAct, tool-calling agents, and autonomous agent platforms (including the OpenClaw and Hermes ecosystems), and implement production-ready systems using modern frameworks (LangGraph, OpenAI Codex, Google Agent Development Kit, NVIDIA Agent Toolkit). The course covers industry-standard interoperability protocols including the Model Context Protocol (MCP), Agent2Agent (A2A), and emerging commerce protocols (ACP/UCP). Students work with both cloud-hosted and locally deployed open models (Gemma 4, Nemotron) to build hybrid inference architectures. The course addresses multi-agent orchestration, retrieval-augmented generation (RAG), human-in-the-loop design, computer use, and browser automation. Throughout, students critically assess challenges in agent security — including prompt injection, data exfiltration, and unauthorized autonomous actions — and apply evaluation methodologies, safety benchmarks, and enterprise deployment practices.

Learning Objectives

By the end of this course, students will be able to:

- Analyze agentic AI architectures, reasoning patterns (ReAct, tool-calling, configurable thinking), and orchestration frameworks to evaluate their suitability for specific applications.

- Implement autonomous agents using production frameworks (LangGraph, OpenAI Codex, Google ADK, NVIDIA Agent Toolkit, OpenClaw, Hermes) with external tool integration, hybrid LLM routing, and persistent memory.
- Design multi-agent systems incorporating RAG pipelines, orchestration patterns (supervisor, hierarchical, sub-agent), and human-in-the-loop capabilities.
- Integrate agent interoperability protocols (MCP, A2A, ACP/UCP) and evaluate compliance with industry standards including the Agentic AI Foundation specifications.
- Deploy production-ready agentic applications across cloud infrastructure and local/edge environments with appropriate monitoring, security guardrails, and failure handling.
- Evaluate agent capabilities and limitations using structured evaluation methodologies, safety benchmarks (PinchBench, OSWorld, WebArena, AgentHarm), and adversarial testing including prompt injection resistance.
- Assess risks in autonomous systems — including prompt injection, data exfiltration, unauthorized actions, and misalignment — and formulate layered mitigation strategies using container isolation, policy-based guardrails, and permission scoping.

Prerequisite(s): None

Recommended Preparation: Python coding on the level of TAC 115; CSCI 571, Web Technologies; CSCI 567, Machine Learning.

Course Notes

Brightspace will be used for hosting the course materials (slides and homework assignments) and grades. Brightspace will be used by students for uploading completed assignments and taking quizzes.

Technological Proficiency and Hardware/Software Required

- **Languages and Tools:** Python is the primary programming language. Students will also use shell/terminal commands (bash), YAML configuration, and TypeScript (for OpenClaw/NanoClaw). Familiarity with Git and GitHub is expected.
- **Cloud Infrastructure:** Google Cloud Run for Assignments 1, 2 and 4; credits will be provided. The NVIDIA Developer Program (free, at build.nvidia.com) provides remote LLM inference via OpenAI-compatible API endpoints.
- **Local LLM Inference:** Assignment 3 requires running a local model. Gemma 4 E4B (~5GB RAM at 4-bit quantization) runs on any laptop from the last 3 years. Students with 16GB+ RAM can run the more capable Gemma 4 26B-A4B. Local inference uses Ollama or llama.cpp — both free and open-source. Reference: a 2018 Mac mini with 32GB RAM runs Gemma 4 E4B comfortably.
- **Hardware:** Laptop or desktop from 2023 or newer recommended. Minimum 8GB RAM; 16GB+ recommended.
- **Software (all free):** Python 3.10+, Git, Docker, Ollama, Node.js/npm, a messaging platform account (WhatsApp, Telegram, or Discord), IDE with AI integration (VS Code with Claude Code or Codex extension recommended).
- **LLM Access (zero cost):** NVIDIA NIM API free tier (~40 RPM, no per-token billing); Gemma 4 locally via Ollama; cloud models (Gemini, GPT, Claude) via free tiers or university-provided access.

Required Readings and Supplementary Materials

- LangGraph: <https://docs.langchain.com/oss/python/langgraph/overview>
- Gemini: <https://ai.google.dev/gemini-api/docs>
- MCP: <https://modelcontextprotocol.io/docs/>
- A2A Protocol: <https://a2a-protocol.org/latest/>
- Linux Foundation Agent2Agent project: <https://github.com/a2aproject>
- OpenAI Agents: <https://openai.github.io/openai-agents-python/>
- Google ADK: <https://google.github.io/adk-docs/>

- AAIF: <https://aaif.io/>
- Microsoft Agent Framework: <https://learn.microsoft.com/en-us/agent-framework/overview/agent-framework-overview>
- Anthropic Claude Agent SDK: <https://www.anthropic.com/engineering/building-agents-with-the-claude-agent-sdk>
- OpenClaw GitHub repo: <https://github.com/openclaw/openclaw>
- OpenClaw: <https://docs.openclaw.ai/start/getting-started>
- NanoClaw: <https://github.com/qwibitai/nanoclaw>
- Hermes Agent: <https://hermes-agent.nousresearch.com/>
- NVIDIA NemoClaw: <https://www.nvidia.com/en-us/ai/nemoclaw/>
- NVIDIA NIM API catalog: <https://build.nvidia.com>
- Gemma 4: <https://deepmind.google/models/gemma/gemma-4/>
- Ollama (for local model deployment): <https://ollama.com/library/gemma4>
- OpenAI Codex: <https://developers.openai.com/codex>
- Claude Code: <https://code.claude.com/docs/en/overview>
- Gemini Enterprise Agent Platform (the rebrand of Vertex AI): <https://docs.cloud.google.com/gemini-enterprise-agent-platform/overview>
- Gemini Spark (Google's 24/7 personal AI agent): <https://gemini.google/overview/agent/spark/>

Weekly slides will reference specific readings and chapters.

Optional Readings and Supplementary Materials

- ReAct (foundational): <https://research.google/blog/react-synergizing-reasoning-and-acting-in-language-models/>
- Toolformer (historical context): <https://ai.meta.com/research/publications/toolformer-language-models-can-teach-themselves-to-use-tools/>
- AutoGPT, BabyAGI (historical context): <https://agpt.co/>, <https://github.com/yoheinakajima/babyagi>
- Voyager (historical context): <https://voyager.minedojo.org/>
- LangChain (still relevant for components): <https://python.langchain.com/docs/introduction/>
- Anthropic's "Building Effective Agents" guide: <https://www.anthropic.com/engineering/building-effective-agents>
- OpenAI's A Practical Guide to Building Agents: <https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf>
- OpenAI Building Agents: <https://developers.openai.com/tracks/building-agents/>
- UK AISI evaluation frameworks: <https://aaif.io/>
- OpenClaw Wikipedia article: <https://en.wikipedia.org/wiki/OpenClaw> (best overview of history, architecture, security concerns)
- "The Six Claws: A Field Guide to Open-Source AI Agent Frameworks" (ibl.ai blog, Feb 2026) <https://ibl.ai/blog/the-six-claws-a-field-guide-to-open-source-ai-agent-frameworks>
- Anthropic "Eight Trends Defining How Software Gets Built in 2026": <https://clade.com/blog/eight-trends-defining-how-software-gets-built-in-2026>
- "AI Agent Protocol Ecosystem Map 2026" (Digital Applied blog) <https://www.digitalapplied.com/blog/ai-agent-protocol-ecosystem-map-2026-mcp-a2a-acp-ucp>
- NVIDIA GTC 2026 keynote (Jensen Huang on OpenClaw, Software 3.0 on YouTube) https://www.youtube.com/live/jw_o0xr8MWU
- Gemma 4 announcement blog: <https://blog.google/innovation-and-ai/technology/developers-tools/gemma-4/>
- Google I/O 2026 AI recap: <https://blog.google/innovation-and-ai/technology/ai/google-ai-updates-april-2026/>
- Code with Claude 2026 conference (Anthropic's developer conference): <https://claude.com/code-with-claude/san-francisco>
- Gartner 2026 Hype Cycle for Agentic AI: <https://www.gartner.com/en/articles/hype-cycle-for-agentic-ai>

Building Agentic AI Systems: Create intelligent, autonomous AI agents that can reason, plan, and adapt by Anjanava Biswas, Wrick Talukdar, Matthew R. Scott (Foreword), Dr. Alex Acero (Foreword), ISBN: 978-1803238753, Packt Publishing, April 21, 2025.

Generative AI with LangChain: Build production-ready LLM applications and advanced agents using Python, LangChain, and LangGraph 2nd Edition by Ben Auffarth, Leonid Kuligin, ISBN: 978-1837022014, Packt Publishing, May 23, 2025.

Note: These texts provide foundational concepts. For current tools and frameworks, refer to the required readings and documentation links above.

Assessments and Assignments

There will be 4 assignments:

Assignment 1: Build a Tool-Using Agent with MCP Integration

Students will build an intelligent agent using LangGraph, OpenAI Agents SDK, or Google ADK with MCP (Model Context Protocol) tool integration. The agent must connect to at least two MCP servers, demonstrate tool-based reasoning and selection, implement conversational memory, and deploy to Google Cloud. Grading covers MCP integration, agent reasoning, memory implementation, and code quality.

Assignment 2: Multi-Agent System with RAG and Orchestration

Students will design a multi-agent system using LangGraph orchestration patterns (Supervisor or Hierarchical). The system must include at least three specialized agents, a RAG pipeline with Pinecone, support for multiple LLMs with dynamic routing, and a web-based frontend. Grading covers orchestration correctness, RAG quality, routing logic, and deployment.

Assignment 3: Personal Assistant with Hybrid LLM Architecture

Students will build and deploy your own OpenClaw or Hermes-based personal AI assistant using hybrid LLM architecture: remote models via OpenAI OAuth, NVIDIA NIM API for complex reasoning, with local Gemma 4 fallback for privacy and offline use. The agent delivers proactive daily briefings, email summaries, and calendar alerts via messaging platforms, with required security guardrails and persistent memory.

Assignment 4: Final Capstone Project

Students will design, develop, deploy, and evaluate an original production-ready agentic AI application — distinct from Assignment 3 — demonstrating advanced features including persistent memory, planning loops, human-in-the-loop controls, and adversarial safety evaluation including prompt injection testing. Deliverables include source code, deployment, demo video, technical presentation, and evaluation report with quantitative metrics.

Each assignment serves to measure student performance on at least one or more learning objectives. Each assignment includes starter code. Deliverables submitted via ZIP on Brightspace.

Process Log Policy: all assignments require a process log (10% of score). AI users: submit “vibe coding” prompts and fixes. Non-AI users: describe your coding process. Missing logs get 0/10.” This is fair and encourages reflection.

Quizzes

Short quizzes will be offered (open-note, 10–15 minutes) to reinforce key concepts. Quizzes will be online using the Brightspace Quiz Tool. These quizzes contribute to participation credit.

Participation

Participation credit (15% of final grade) is earned through active engagement in class discussions, completion of in-class activities, and timely submission of weekly quizzes. Students must contribute meaningfully to discussions and complete all assigned quizzes to receive full participation credit.

Grading Breakdown

Assessment	% of Grade
Assignment 1	15
Assignment 2	15
Assignment 3	20
Final Capstone Project	35
Participation / Reading Discussions	15
TOTAL	100

Final Capstone Project Rubric (35%)

- Technical Implementation (correctness, complexity): 40%
- Originality and Design: 20%
- Evaluation and Results: 15%
- Presentation Clarity and Demo: 15%
- Safety, Ethics, or Alignment Consideration: 10%

Grading Scale

Grades are not curved but based on proficiency with the material. Ranges operate in favor of the students. At the end of the semester, if the average in the class is lower than 80%, the average will become the cut-off between a B- and a C+. Final scores will be rounded to the nearest whole number (.50 and above rounds up).

$x \geq 93$	A	$73 \leq x < 77$	C
$90 \leq x < 93$	A-	$70 \leq x < 73$	C-
$87 \leq x < 90$	B+	$67 \leq x < 70$	D+
$83 \leq x < 87$	B	$63 \leq x < 67$	D
$80 \leq x < 83$	B-	$60 \leq x < 63$	D-
$77 \leq x < 80$	C+	$x < 60$	F

Course Policies

Assignment Submission

Assignments will be discussed in class and worked on individually. Discussion among students is fine, but no copying of other student's code is allowed. Except for Assignment 3, Personal Assistant, all assignments and final project need to run online on Google Cloud, and grading will only occur if the applications are able to run online. Except for Assignment 3, assignments running on "localhost" will not be acceptable. Assignments source code and links to the online apps will be submitted to Brightspace (instructions will be provided in class and on Piazza) and are due by 11:59p.m. on the due date (see Late Policy below). Grading criteria (aka Rubrics) will be provided with the assignment description. All grades will be posted to Brightspace gradebook.

Regrading

Once grades are entered into Brightspace gradebook, students will be able to request a regrade if they think a mistake has been made by the grader through the following process:

1. Once an assignment is graded, an online form will be provided on Piazza, to submit a formal regarding request. Note that this is the only request that can be made to regrade that specific assignment, so be sure to include all relevant information. If the request is submitted more than 2 days after form is posted, the request will be denied.

2. The TA will review the request and determine if a regrade will be granted.
 - a. If the regrade request is denied, the original grade will stand.
 - b. If the regrade request is granted, the TA will forward the request to a grader (possibly a different one than who originally graded it). The grader will conduct a regrade and send the updated score to the TA, who will then enter it, with feedback, into Brightspace.
3. There will be **only one regrade request per assignment**, using an online form, which may include regrade of multiple items, and the score after the regrade is final. If any questions arise beyond that, the student will need to speak with the professor in person.
4. There is no regrade of the final project, due on Final Week.

Late Submission

Each student will have **two (2) grace days** to use during the semester for submitting assignments late. Grace days can only be used on *assignments* (**not** final project, exams, etc.), and the days can be used in any combination. For example, you could use one grace day on assignment 1 and one grace day on assignment 2. All submissions other than for assignments in accordance with this policy must be submitted by 11:59p.m. on the due date or will receive a 0.

After the two grace days have been used, any assignment submitted late will receive a 0. A grace day will be counted for any assignment submitted after 11:59p.m. on the due date. To state that another way, if an assignment is submitted after the due date, at 12:00a.m. (midnight) or later, grace days will be used. The grace days do not need to be approved by the professor, but any exceptions other than grace days will need professor approval. In exceptional circumstances, late submissions will be accepted, with a 30% penalty.

Attendance

Attendance will not be taken. Students who do not attend the lectures are responsible for all material covered in the lectures. The course uses the Piazza online forum platform to efficiently stimulate and manage class discussions. Discussions are limited to course materials, assignments, and exams. Following all postings on Piazza is also required.

Academic Integrity

The University of Southern California is foremost a learning community committed to fostering successful scholars and researchers dedicated to the pursuit of knowledge and the transmission of ideas. Academic misconduct is contrary to this fundamental mission and includes any act of dishonesty in the submission of academic work (either in draft or final form), as well as cheating, plagiarism, fabrication (e.g., falsifying data), knowingly assisting others in acts of academic dishonesty, and any act that gains or is intended to gain an unfair academic advantage. Students are expected to uphold the highest standards of academic integrity in all coursework. This course follows the expectations for academic integrity as stated in the [USC Student Handbook](#). All students are expected to submit assignments that are original work and prepared specifically for the course/section in this academic term. Students may not submit work written by others or “recycle” work prepared for other courses without obtaining written permission from the instructor(s). Students suspected of academic misconduct will be reported to the Office of Academic Integrity.

Academic dishonesty has a far-reaching impact and is considered a serious offense against the university. Violations will result in a grade penalty, such as a failing grade on the assignment or in the course, and disciplinary action from the university, such as suspension or expulsion. For more information about academic integrity see the [Student Handbook](#), the [Office of Academic Integrity's website](#), and university policies on [Research and Scholarship Misconduct](#). Please ask your instructor if you are unsure what constitutes unauthorized assistance on an exam or assignment or what information requires citation and/or attribution.

Instructors determine whether and how generative AI tools (such as ChatGPT, Copilot, Claude, Gemini, and similar applications) and other external resources (such as English-language translation tools) may be used

in this course. Expectations may vary from assignment to assignment. This includes AI tools that record, transcribe, or summarize class sessions, which may require express instructor permission. Review the AI use guidance provided below carefully and direct any questions to your instructor before submitting work.

Required: Students seeking general guidance may consult the following resources:

- [USC Student Handbook](#)
- [Generative AI General Policy](#)
- [Practical Tips for Using AI in Academic Work](#)
- [Office of Academic Integrity](#)

Permitted Use of Artificial Intelligence (AI) Tools

Generative AI is encouraged: students are expected to use AI (e.g., IDEs like VS Code, AI Assistants like GitHub Copilot and LLMs like GPT-5.5) in this class. The ability to use artificial intelligence (AI) tools effectively and responsibly is an increasingly important professional skill. In this course, you are encouraged to use AI tools to support your learning and coursework.

When using AI tools, keep the following in mind:

- AI may be used to support activities such as brainstorming, exploring ideas, generating examples, revising drafts, summarizing information, coding assistance, or other learning-related tasks, as appropriate.
- You remain responsible for the accuracy, quality, and integrity of all work you submit. AI-generated content may be incomplete, inaccurate, biased, or fabricated and should be reviewed and verified before use.
- AI should be used to support your learning, not replace the intellectual work required to meet the course learning objectives.
- Do not upload personal, confidential, proprietary, or otherwise sensitive information to AI tools unless specifically authorized. Be mindful that information entered into some AI systems may be retained or used in ways outside the university's control.
- Respect copyright, licensing, and intellectual property rights when using AI-generated or AI-assisted content. These materials may not be free of third-party rights or suitable for unrestricted reuse.
- Any use of AI in course assignments does not have to be disclosed and appropriately attributed.
- Additional instructions regarding AI use may be provided for individual assignments or assessments, such as submission of AI logs.

Course Evaluations

Course evaluation occurs at the end of the semester university wide. It is an important review of students' experience in the class. The intent of the end-of-semester evaluation is to provide an evaluation of the course, the instructor, and whether the learning objectives have been achieved. In addition, the instructor will provide an online [mid-semester evaluation](#) that will be used for early course correction.

Course Schedule

	Topics	Readings and Preparation	Deliverables
Week 1	Introduction to Agentic AI / Foundations / AI-Native Development & Vibe Coding / Software 3.0	Course overview; 2026 agent landscape; Karpathy's Software 1.0 → 2.0 → 3.0 framework; OpenClaw's repository, runtime workspace, skills, channels, and optional isolation; the "Agent Manager" role; Agentic AI Foundation (AAIF), AGENTS.md, and A2A. IDE + AI Assistants; LLM-assisted development workflow; AGENTS.md and repository-level AI instructions; Vibe coding with Claude Code and OpenAI Codex; NVIDIA GTC 2026 keynote (OpenClaw/NemoClaw sections); OpenClaw Wikipedia article	
Week 2	LLMs as Agents / Prompting Techniques / Local & Cloud Models	LangGraph documentation: Introduction and Getting Started; cloud models (Gemini, GPT, Claude) and local models (Gemma 4 via Ollama) as swappable agent backends; model selection by tool-call reliability, latency, context, cost, and local-vs-cloud tradeoffs. Prompting techniques including zero-shot, few-shot, chain-of-thought, configurable thinking (GPT-5.5 reasoning effort; Gemma 4 think token), and ReAct	
Week 3	Planning & Reasoning Loops / ReAct, CoT / Planning patterns and LangGraph	ReAct: reasoning-and-acting loops; configurable thinking and Chain-of-Thought; building ReAct from scratch with tool calls, observations, and bounded iteration; message-shape mechanics, multiple tool calls per model round-trip, context growth, and agent failure modes. Self-consistency and Tree-of-Thought for sampling and searching reasoning paths. Plan-and-Execute and Reflexion for long-horizon planning, replanning, and learning across runs. LangGraph orchestration, supervisor and hierarchical patterns, pattern composition, and production failure modes.	
Week 4	Tool Use / MCP / OpenClaw Skills / LangGraph Tools / A2A / Webhooks / Commerce Protocols	MCP and tool integration: JSON-RPC 2.0, 2025-11-25 compatibility lifecycle and 2026 stateless MCP; stdio/HTTP transport. OpenClaw skills as a sibling architecture. LangGraph Tools above MCP. NVIDIA NIM as an OpenAI-compatible inference endpoint. A2A v1.0 (Agent-to-Agent) — Agent Card discovery, messaging, task lifecycle, artifacts, and push notifications. Webhooks — A2A delivery; HMAC signing, retries, and dead-letter. Commerce protocols (AP2, ACP, UCP) — landscape only.	

Week 5	Memory Architectures / LangGraph / OpenClaw Workspace Files / Context Windows	LangGraph memory; OpenClaw workspace files (SOUL.md, USER.md, IDENTITY.md); Claude Managed Agents “Dreaming” for autonomous memory consolidation; OpenAI Dreaming for background memory curation; Gemini API documentation. Gemma 4 128K–256K context windows; long-context retrieval without RAG or external memory stores; Ollama local inference and hardware/model selection; KV cache, MoE, quantization, and long-context trade-offs.	Assignment 1 Released
Week 6	Multi-Agent Orchestration / Communication and Collaboration	LangGraph for multi-agent workflows; orchestration patterns: Supervisor, Collaboration, Hierarchical teams, Plan-and-execute, Swarm; LangChain subagents, handoffs, routers, and custom workflows. Claude Code sub-agents — scoped tools, models, and delegation; Codex CLI parallel agents with isolated Git worktrees; OpenClaw experimental Swarm orchestration; Claude Managed Agents — hosted multi-agent orchestration, Outcomes rubric-based evaluation, and Dreaming memory consolidation.	
Week 7	Case Studies: Production Agent Architectures	OpenClaw: self-wired architecture, Gateway, memory, and skill ecosystem; ClawHub and SKILL.md/Agent Skills standard; OpenClaw + NVIDIA security collaboration — SkillSpector, Security Signals dataset, cryptographically signed Skill Cards; NemoClaw enterprise sandboxing and policy-based guardrails. Production agent architectures and outcomes: Gemini Spark personal agents; Claude Code and Codex at Rakuten; TELUS Fuel iX and Zapier at scale; Klarna and Elastic production cases. Governance and autonomy lessons from MoltMatch; self-wired, platform-owned, and hybrid architectures in practice.	
Week 8	Modern Agent Architectures & SDKs / The Claw Ecosystem	OpenAI Codex: desktop, CLI, IDE, Skills, Automations, and SDK; Google ADK 2.x with Gemini 3.6 Flash and MCP; NVIDIA Agent Toolkit: OpenShell, NemoClaw, AI-Q, Nemotron; Claude Agent SDK and sub-agents; Microsoft Agent Framework; LangGraph model-agnostic orchestration; AGENTS.md, MCP, and A2A convergence. OpenClaw architecture: Gateway, skills, workspace, messaging, security; Original NanoClaw: minimal host, Claude Agent SDK, container isolation; NanoClaw v2 evolution; Voyager as historical architecture lineage.	Assignment 1 Due / Assignment 2 Released

Week 9	Agent Security / AI Ethics	Agent security: prompt injection, exfiltration, consent, and Codex command injection; NanoClaw containers, NemoClaw/OpenShell policy enforcement, IronClaw WASM, ZeroClaw lightweight controls; OpenClaw hardening; Anthropic Project Glasswing/Claude Mythos. Benchmarks: PinchBench (NEW; OpenClaw-specific), OSWorld, WebArena, AgentHarm, HELM Safety, AIR-BENCH Live, TrustLLM; framework: ControlArena (UK AISI); industry index: AI Safety Index (Future of Life Institute).	
Week 10	Protocol Mechanics — JSON-RPC, WebSockets, Streaming	JSON-RPC 2.0 — requests, responses, errors, notifications, batching, and MCP tool calls. MCP 2026-07-28 — stateless protocol model, server/discover, tool schemas/results, stdio and Streamable HTTP, and header-based routing (Mcp-Method, Mcp-Name). WebSockets — handshake, persistent full-duplex channels, custom JSON-RPC messaging. LLM text-delta streaming over HTTP/SSE and WebSocket. A2A v1.0 — task model, SSE/gRPC streaming, Linux Foundation governance, 150+ supporting organizations.	Assignment 2 Due / Assignment 3 Released
Week 11	Computer Use & Autonomous Agents — Capability, Orchestration, Safety, and Evaluation	The capability layer. Computer use as agents acting on browser and desktop UI surfaces: OpenAI’s computer use tool and ChatGPT Work, Anthropic Computer Use, the Browser Use framework, and Codex native computer use. Perception–reasoning–action loops; visual, DOM, and CDP-based action surfaces; orchestration and autonomy levels; guardrails, human oversight, and benchmark-driven evaluation. The orchestration layer. Loops as the unit of autonomous work—from ReAct, AutoGPT, and Ralph to goal-oriented agents and multi-agent orchestration. Loop economics, verification, and halting controls: iteration and time limits, cycle detection, and budget ceilings. Production loops as durable agents, with skills as reusable components and human oversight at consequential decisions.	
Week 12	Human-in-the-Loop Systems / Feedback and Control	RLHF foundations and humans-in- vs. humans-above-the-loop; durable LangGraph approvals with interrupt() and Command(resume=...); approve, reject/reroute, and edit patterns; Claude Code permission modes and Auto Mode; OpenAI Codex sandboxing and approvals. Claude Managed Agents: Outcomes, memory/Dreaming, multiagent orchestration, and reasoning-blind evaluation; EU AI Act Articles 14 and 26 with the amended 2026–2028 timeline; NIST AI RMF and AI Agent Standards Initiative; Enterprise Automation: Revionics multiagent pricing with Google Cloud ADK.	Final Capstone Project Released

Week 13	Agent UX / Messaging-Native and Generative Interfaces	Messaging-native UX across OpenClaw’s Telegram, Discord, terminal, Control UI, and mobile surfaces; Slack interaction and approval patterns; workspace-first control in Codex and Gemini Spark; AG-UI for interoperable agent-to-application events; and A2UI/Open-JSON-UI for declarative agent-generated interfaces, including commerce and browser-automation failure modes. Generative UI and agent commerce: A2UI, MCP Apps, and AG-UI; Operator’s “\$31 eggs” UX failure and Agentic Commerce Protocol; browser automation with major service platforms; legacy-system automation without APIs; OpenClaw demo using WebSockets and SSE.	
Week 14	Final Capstone Project Discussion / Deep Research + Agentic RAG	Assignment 4 Final Capstone — original, production-ready agentic application, complexity $\geq$ Assignment 3 (not an extension). Solo or teams of up to three implement ≥ 3 of seven features: memory, planning, HITL, computer use, A2A, agentic RAG, or hybrid inference. Requires guardrails, evaluation, adversarial testing, deployment, six deliverables, and per-member process logs. Multi-step research loops. Four Deep Research products: OpenAI, Gemini, Perplexity, and GPT-Researcher. Three RAG patterns: 2-step, agent-controlled, and iterative/hybrid. Query decomposition, source verification, evidence evaluation, refinement, termination, and cited synthesis.	Assignment 3 Due
Week 15	Guest Lecture / The Future of Agentic AI	Guest lecture (topic TBD). June 30, 2026 synthesis of eight independently queried frontier models on agentic AI’s 12-month outlook. Examines execution-centric capabilities, protocol taxonomy and early scorecards, hype casualties, evaluation and governance infrastructure, deployment obstacles, and falsifiable predictions—closing the course arc with Karpathy’s Software 3.0 framing.	
FINAL	Final Capstone Project Presentations		Refer to the final exam schedule in the USC <i>Schedule of Classes</i> at classes.usc.edu .

Assessment Details (Appendix)

Assignment 1: Build a Tool-Using Agent with MCP Integration

Overview

Students will build an intelligent agent using LangGraph (or their choice of OpenAI Agents SDK / Google ADK) with MCP (Model Context Protocol) tool integration. The agent must connect to at least two external tools via MCP servers and demonstrate planning, tool selection, and conversational memory.

Requirements

- **Framework:** LangGraph, OpenAI Agents SDK, or Google ADK (student choice)
- **LLM:** Any supported cloud model (Gemini, GPT, Claude models, Nvidia NIM)
- **Skills:** OpenClaw's MCP-style tools integrations, study ClawHub for inspiration
- **Tools:** Integrate 2+ MCP servers (e.g., filesystem, web search, weather, database)
- **Memory:** Implement conversational memory using framework's built-in capabilities
- **Deployment:** Google Cloud Run

Deliverables

- Python source code with MCP server configurations
- Working deployment URL on Google Cloud
- Process Log documenting AI-assisted development
- README with setup instructions and architecture diagram

Grading

- MCP integration correctness (25%)
- Agent reasoning and tool selection (25%)
- Memory implementation (20%)
- Code quality and documentation (20%)
- Additional MCP servers or custom tool creation (10%)

Assignment 2: Multi-Agent System with RAG and Orchestration

Overview

Students will design and deploy a multi-agent system using LangGraph's orchestration patterns (Supervisor, Hierarchical, or Collaboration). The system must include a RAG pipeline with Pinecone, support for multiple LLMs with dynamic routing, and expose capabilities via a web interface.

Requirements

- **Architecture:** Multi-agent system with at least 3 specialized agents, reference architecture: Claude Code sub-agent patterns
- **Orchestration:** Implement Supervisor or Hierarchical pattern using LangGraph, or Codex's worktree-based multi-agent model
- **LLMs:** Integrate 2+ different LLMs with intelligent routing (e.g., cost-based, capability-based); same LLM options as Assignment 1
- **RAG:** Pinecone vector store with document ingestion pipeline
- **Frontend:** FastAPI + SSE + vanilla HTML/CSS/JS interactive UI (recommended); FastAPI + HTMX also permitted
- **Deployment:** Google Cloud Run with proper environment management; NemoClaw/OpenShell

Suggested Agent Specializations (pick 3+)

- Research Agent (web search + RAG retrieval)
- Code Agent (code generation/analysis)
- Data Agent (structured data queries)
- Summary Agent (document summarization)

- Router/Supervisor Agent (orchestration)

Deliverables

- Complete source code with multi-agent architecture
- Live deployment URL
- Architecture documentation showing agent communication flow
- Evaluation results (latency, accuracy, cost per query)
- Process Log

Grading

- Multi-agent orchestration correctness (25%)
- RAG implementation quality (20%)
- LLM routing logic (15%)
- Frontend usability (15%)
- Deployment and scalability (15%)
- A2A protocol integration or MCP tool sharing between agents (10%)

Assignment 3: Personal Assistant with Hybrid LLM Architecture

Overview

Build, configure, and deploy your own OpenClaw or Hermes-based personal AI assistant that acts as a proactive digital chief of staff. Using a hybrid LLM architecture (remote LLM via NVIDIA NIM API + local LLM fallback via Gemma 4), the assistant doesn't just respond to commands — it anticipates needs, delivers a daily briefing, monitors email, and keeps the student organized.

The assistant knows your schedule, your priorities, and your preferences, and surfaces what you need before you ask for it. This assignment integrates the course's core themes — tool use, memory, safety, and human-in-the-loop design — into a single, personally meaningful project that students can continue using after the course.

Requirements

- **Core Platform:** Core Platform: OpenClaw or Hermes (either platform is acceptable; NanoClaw acceptable for students prioritizing security)
- **Primary LLM:** Any model via NVIDIA NIM API, specialized in agentic workflows (e.g., Nemotron 3.5 Lightning, DeepSeek V4 Flash, Laguna XS, Kimi K3) — free tier
- **Local Fallback LLM:** Gemma 4 E4B (minimum) or 26B-A4B (recommended if hardware supports it), running via Ollama or llama.cpp on the student's own machine
- **Messaging Interface:** Working integration with at least one messaging platform (Telegram, Discord, or Signal)

Core Features

- **Skills Integration:** Skills Integration: Create or install at least 4 skills — at minimum: calendar, email, web search, and one custom skill of the student's choosing. Both OpenClaw and Hermes ship with similar built-in skills (many overlap by name and interface) (e.g., news aggregation, file organization, code repository monitoring, Spotify/music control, smart home)
- **Security:** Implement at least 2 security measures:
 - Container isolation (NanoClaw-style) OR OpenShell-style policy guardrails
 - Input validation / prompt injection defense
 - Permission scoping (what the agent can and cannot do — e.g., can read email but cannot send without approval)
 - Credential management (API keys stored securely, not in plaintext config)
- **Memory:** Memory: Configure persistent memory across sessions using the platform's workspace files (OpenClaw's SOUL.md, USER.md, IDENTITY.md; Hermes equivalents; or a comparable persistent

workspace representation). The agent should remember the student's preferences, prior interactions, and ongoing tasks between sessions.

- **Scheduling/ Heartbeat:** Implement a cron-based or heartbeat scheduling mechanism so the agent runs proactively (morning briefing, periodic email checks) without requiring the student to initiate every interaction. This is the key difference from Assignments 1 and 2 — the agent acts on its own schedule.

Advanced Features (implement at least 3)

- **Morning Briefing:** A scheduled daily message delivered via the messaging platform each morning, including:
 - Today's calendar events and schedule gaps
 - Weather forecast for the student's location
 - Top unread/flagged emails with one-line summaries
 - Optionally: top news headlines relevant to the student's interests
 - Optionally: reminders for upcoming assignment deadlines
- **End-of-Day Recap:** An evening summary message including
 - What the agent did during the day (tasks completed, messages handled)
 - Items flagged for human review or decision
 - Tomorrow's preview (early meetings, deadlines approaching)
- **Email Integration:** Connect to the student's email (Gmail via API or MCP server) to:
 - Summarize incoming emails on request or proactively
 - Draft replies for review (never send without approval — human-in-the-loop)
 - Flag urgent messages for immediate notification
- **Proactive Alerts:** Context-aware notifications such as:
 - "You have a meeting in 30 minutes — want me to pull relevant context?"
 - "Assignment 4 is due in 3 days — your repo has uncommitted changes"
 - "You haven't responded to [sender]'s email from yesterday — it looks time-sensitive"
- **Calendar Intelligence:** Beyond reading the calendar:
 - Detect scheduling conflicts
 - Suggest optimal times for tasks based on schedule gaps
 - Prepare meeting context from prior emails/documents
- **Personal Knowledge Base:** The agent learns the student's preferences over time:
 - Preferred communication style
 - Commonly referenced people, projects, or topics
 - Frequently used tools and workflows

Deliverables

- Complete source code including OpenClaw/Hermes/NanoClaw platform configuration, custom skills, routing logic, and scheduling setup
- Working deployment running on the student's own hardware (Mac, Linux, or Windows) — must demonstrate at least 48 hours of continuous operation
- Security audit document: threat model identifying at least 5 risks, mitigations implemented, known remaining risks
- Performance comparison report: response time, quality, and cost analysis between remote (NIM) and local (Gemma 4) models across different task types
- Personalization Profile: the workspace configuration (SOUL.md/USER.md for OpenClaw, equivalent for Hermes) showing how the student personalized their assistant
- Process Log documenting AI-assisted development
- Demo video (5–7 minutes) showing:
 - The morning briefing being delivered
 - The agent responding to a real-time request via messaging
 - The hybrid routing in action (demonstrating both remote and local model use)
 - At least one proactive alert
 - The end-of-day recap (can be simulated if timing doesn't align with demo recording)

Grading

- Hybrid LLM routing correctness and intelligence (20%)
- Proactive features — quality and usefulness of briefings, alerts, and recaps (15%)
- Security implementation and audit quality (15%)
- Skill integration and overall agent utility (15%)
- Memory, personalization, and scheduling (15%)
- Code quality, documentation, and demo (10%)
- Any (1) of the following: NemoClaw/OpenShell integration; custom skill published to ClawHub; A2A integration with another student's personal assistant (e.g., shared calendar coordination); integration with 3+ messaging platforms simultaneously (10%)

Assignment 4: Final Capstone — Production-Ready Agentic Application

Overview

Design, develop, deploy, and evaluate an original production-ready agentic AI application. The capstone is worth 35% of the final course grade and may be completed individually or in a team of up to three students. The same criteria and weights apply to every submission; expected scope is proportional to team size.

The project must meet or exceed Assignment 3 complexity but may not be an extension of the Assignment 3 Personal Assistant. Architectural lessons may carry forward; the application and design work must be new.

Requirements

Advanced Features (implement at least 3)

- Long-term memory (cross-session persistence)
- Multi-step planning with self-correction
- Human-in-the-loop approval workflows
- Computer use or browser automation
- Multi-agent collaboration with A2A protocol
- Agentic RAG with iterative retrieval and source verification
- Hybrid cloud-local inference with intelligent routing

Submissions with fewer than three working advanced features receive partial Technical Implementation credit for verified work; they do not receive an automatic zero for the entire criterion.

Safety & Evaluation

Every capstone must include both a safety subsystem and an evaluation subsystem.

- Implement layered guardrails appropriate to the actual attack surface, such as input validation, output filtering, rate limiting, permission scoping, execution sandboxing, or network policy.
- Attempt at least three prompt-injection attacks against the deployed system. Document the injection point and prompt, the actual trace or response, and the mitigation or known limitation for each attack.
- Quantitatively evaluate the system using a capability-matched benchmark (PinchBench, OSWorld, WebArena, or AgentHarm), LangSmith or AgentOps metrics, or a custom evaluation harness with stated methodology.
- Document significant failure modes and mitigations.
- Doc NemoClaw/OpenShell and Codex Security are optional reference tools, not required implementations.

Deployment

Choose the topology that fits the application:

1. Google Cloud Run with monitoring
2. Local/edge deployment with NemoClaw/OpenShell
3. OpenClaw/Docker

4. Hermes-based local deployment

Every deployment path must produce a live URL that TAs can reach. Local deployments must use Tailscale as the approved remote-access path. Long-term memory on Cloud Run requires Firestore or an equivalent external durable store; in-process state is not durable.

Deliverables

Six deliverables are submitted through Brightspace:

1. Complete source code as a GitHub repository link or ZIP, with README
2. Live deployment URL reachable by the TAs
3. Demo video (5–10 minutes MP4) showing an end-to-end run, at least one advanced feature, and one guardrail engaging
4. Technical presentation (10+ slides) covering architecture and design decisions, capabilities and limitations, safety and guardrails, evaluation results and metrics, and future improvements
5. Evaluation report (PDF or Markdown) containing quantitative metrics, sample size and date, at least three adversarial tests, failure modes, mitigations, and cost analysis
6. PROCESS_LOG.md, 200–300 words per team member

Deliverables 1–5 are shared by a team. Every team member submits an individual process log. A missing process log deducts 10 percentage points from that student's Assignment 4 score.

Grading

- Technical Implementation (correctness, complexity) (40%)
- Originality and Design (20%)
- Evaluation and Results (15%)
- Presentation Clarity and Demo (15%)
- Safety, Ethics, or Alignment Consideration (10%)

Statement on Academic Conduct and Support Systems

Students and Disability Accommodations

USC welcomes students with disabilities into all of the University's educational programs. [The Office of Student Accessibility Services](#) (OSAS) is responsible for the determination of appropriate accommodations for students who encounter disability-related barriers. Once a student has completed the OSAS process (registration, initial appointment, and submitted documentation) and accommodations are determined to be reasonable and appropriate, a Letter of Accommodation (LOA) will be available to generate for each course. The LOA must be given to each course instructor by the student and followed up with a discussion. This should be done as early in the semester as possible as accommodations are not retroactive. More information can be found at osas.usc.edu. You may contact OSAS at (213) 740-0776 or via email at osasfrontdesk@usc.edu.

Student Financial Aid and Satisfactory Academic Progress

To be eligible for certain kinds of financial aid, students are required to maintain Satisfactory Academic Progress (SAP) toward their degree objectives. Visit the [Financial Aid Office webpage](#) for undergraduate- and graduate-level SAP eligibility requirements and the appeals process.

Support Systems

[Counseling and Mental Health](#) - (213) 740-9355 – 24/7 on call

Free and confidential mental health treatment for students, including short-term psychotherapy, group counseling, stress fitness workshops, and crisis intervention.

[988 Suicide and Crisis Lifeline](#) - 988 for both calls and text messages – 24/7 on call

The 988 Suicide and Crisis Lifeline (formerly known as the National Suicide Prevention Lifeline) provides free and confidential emotional support to people in suicidal crisis or emotional distress 24 hours a day, 7 days a week, across the United States. The Lifeline consists of a national network of over 200 local crisis centers, combining custom local care and resources with national standards and best practices. The new, shorter phone number makes it easier for people to remember and access mental health crisis services (though the previous 1 (800) 273-8255 number will continue to function indefinitely) and represents a continued commitment to those in crisis.

[CARE-SC: Confidential Advocacy, Resources, and Education Support Center](#) - (213) 740-9355(WELL) – 24/7/365 on call.

Confidential advocates, prevention educators, and professional counseling teams work to promote a universal culture of consent, and prevent and respond to gender- and power-based harm. Services available to all USC students at no cost.

[Office of Civil Rights Compliance](#) - (213) 740-5086

Information about how to get help or help someone affected by harassment, discrimination, retaliation on the basis of a protected characteristic, rights of protected classes, reporting options, and additional resources for students, faculty, staff, visitors, and applicants.

[USC Campus Support and Intervention](#) - (213) 740-0411, uscsupport@usc.edu

Assists students and families in resolving complex personal, financial, and academic issues adversely affecting their success as a student.

[USC Emergency Information](#)

Latest updates regarding safety, including ways in which instruction will be continued if an officially declared emergency makes travel to campus infeasible.

[USC Department of Public Safety](#)

For 24 hour emergency assistance or to report a crime: UPC: (213) 740-4321, HSC: (323)-442-1000.
For 24 hour non-emergency assistance or information: UPC: (213) 740-6000, HSC: 323-442-1200.

[Office of the Ombuds](#) - (213) 821-9556 (UPC) / (323-442-0382 (HSC)

A safe and confidential place to share your USC-related issues with a University Ombuds who will work with you to explore options or paths to manage your concern.

[Occupational Therapy Faculty Practice](#) - (323) 442-2850 or otfp@med.usc.edu

Lifestyle Redesign services for USC students to support health promoting habits and routines that enhance quality of life and academic performance.

[The Office of Sustainability](#) promotes numerous ways for all Trojans to study, work, and live sustainably at USC. Students can access tools, resources, and opportunities on their website and in the Sustainability Hub in the Student Union Building, Suite 101.

[USC Report & Response](#) provides a front door to reporting concerns, including an ability to report anonymously, which violate university policies or go against our Unifying Values and helps connect you with the support you need. The website provides a list of confidential resources, supporting offices, and policies that are available to our university community. The [Resources page](#) includes information regarding the university's response in the event of immigration enforcement activity on campus or involving a student or employee. Additional questions may be directed to the Office of Professionalism and Ethics (OPE) and ope@usc.edu or in reviewing the [Office of Professionalism and Ethics](#) website.